PRD to PR Playbook

How a Confluence PRD becomes a reviewed pull request with test evidence, with Claude Code, across the Airflow data repo, the micro-frontends and the frontend shell. Everything described here exists in the `prd-to-pr-rule-skills` repository and was tested; section 18 lists what still needs checking in your environment. The interactive pages cover the same ground with diagrams and charts.

Owner: Engineering Platform | Source: github.com/amarkum/prd-to-pr-rule-skills | Supersedes the draft AI-Native Engineering Implementation Plan

1

The recommendation

1. **Use Claude Code as the platform.** It already provides sandboxing, permissions, subagents, hooks, MCP connectors, plan mode, worktrees and headless CI. Package the process as one versioned plugin marketplace that every repo enables. Build a custom orchestrator later, and only if volume demands it.
2. **Follow the six-stage loop.** Plan, Design, Build, Test, Deploy and Maintain, as in Anthropic's AI-native SDLC playbook. Each stage commits a file to `specs/<KEY>/` that the next stage reads: `intent.md`, `spec.md`, `plan.md`, the evidence, the PR. The git history is the audit trail.
3. **People decide at the gates; hooks enforce them.** The product owner accepts the intent, a human approves the spec, the engineer (or the tech lead for risk: high) accepts the plan, code owners merge and the release manager authorises production. Agents can't make any of these decisions: the hooks refuse.
4. **One builder, specialised per repo.** Each repo declares its stack and commands in `.ai/repo.yaml`; the Airflow, frontend or MFE skill loads automatically by file path.
5. **Evidence is computed, not claimed.** Tests are named after acceptance criteria, new tests must fail without the change, the diff must match the plan, locked tests can't change, and CI re-runs and attests it all.
6. **Start with one repo per stack and measure.** Two weeks on setup 1, compared with the last 20 comparable stories, before automating anything. Lead time, rework and escaped defects are the measures.

2

The loop and its gates

Building is no longer the slow part of delivery; the human-speed steps around it are. The loop keeps those steps but moves them onto files the agent and the people both read, and puts a deterministic check behind every decision that must hold.

STAGE	SKILLS	COMMITS	WHO DECIDES	ENFORCED BY
1 Plan	`prd-review`	`prd.md`, `prd-review.md`, `intent.md`	Product owner: PRD READY and marked ready	`prd_lint.py` and the rubric; questions go back as a Confluence comment and a Jira task
2 Design	`context-gather`, `spec-write`	`context.md`, `spec.md`	A human approves the spec; tech lead for risk: medium or high	Edit hook: agents can't write `status: approved`; no code until it's set
3 Build	`plan`, `build`	`plan.md`, tests, code	The engineer accepts the plan in plan mode; tech lead for risk: high	Edit hook: no code before the plan is accepted; locked tests can't be edited
4 Test	`verify`	`.ai/evidence/` (CI artifact)	A check: evidence PASS for the head commit	PR hooks; `ai-evidence` required check, attested
5 Deploy	`pr-open`, `pr-babysit`	Draft PR, review findings	Code owners merge; release manager authorises production	Hooks: draft PRs only, no merge, release commands need `RELEASE_APPROVAL`; branch protection
6 Maintain	`triage`, `learn`	Incident `intent.md`, `retro.md`, knowledge, evals	Service owner triages: fix now, schedule, dismiss	`bands.py` tiers set the tools Claude gets

The orchestrator skill, `/sdlc-core:prd-to-pr`, runs the stages in order, stops at every gate and resumes from whatever `specs/<KEY>/` already holds. In CI the same stages are triggered by the artifacts themselves: a labelled PRD starts intake, an accepted intent starts design, an approved spec starts the build.

3

Changes to the draft plan

The draft plan had the right instincts: a staged pilot, approval gates, untrusted input, no automatic merge, an evidence bundle and a definition of done. It invested in infrastructure before proving the workflow, and left the artifacts undefined.

TOPIC	DRAFT PLAN	THIS PLAYBOOK	WHY
Build path	16 weeks; orchestrator, workflow engine and six agent services first	A Claude Code plugin from week 1; the artifacts in git hold the workflow state; an orchestrator later, if needed	The first real PR arrives in weeks, not months
Process	A linear pipeline	The six-stage loop, with incidents and drift re-entering as intents	Maintenance starts work itself instead of waiting for someone

TOPIC	DRAFT PLAN	THIS PLAYBOOK	WHY
PRD validation	"Completeness validator"	Deterministic lint plus a rubric review; questions to the product owner with suggested defaults	One pass of consistent feedback
Repo knowledge	Manifests, architecture map	`CLAUDE.md` under a page, `.ai/repo.yaml`, schema snapshots	One file for people, one for machines
Agents	Six agent services	Five subagents and sixteen skills in one runtime	Cheaper to change; separation of duties by tool limits
Evidence	Bundle in the PR	AC coverage, fail-to-pass, plan match, locked tests, hashed manifest, CI re-run, attestation	Green CI can hide weak tests
Learning	"Reusable solution library"	A promotion ladder from knowledge entry to CLAUDE.md line, rule, check and eval	Repeated review comments become checks

4

What goes where

Ask two questions about a piece of guidance: must it hold every time, and who needs it, when? Advisory mechanisms (CLAUDE.md, rules, skills, subagents) make the right thing likely. Enforced mechanisms (hooks, scripts, CI checks, policy) make the wrong thing impossible or visible.

MECHANISM	PUT HERE	KEEP OUT	LOADS
CLAUDE.md	Commands with healthy output, architecture, conventions, things Claude gets wrong, how to verify	Org rules, procedures, secrets	Every session; keep it under a page
Always-on rules	About 50 lines: the workflow contract, security and data, the test standard	Procedures and examples	Session start (SessionStart hook; `.claude/rules/` when vendored)
Skills	Know-how for one kind of task: PRD review, spec, plan, verify, the stack conventions	Always-on rules, one-off instructions	Description always; body when used or by path
Subagents	Work needing its own context or tool limits: analyst, investigator, builder, QA, reviewer	Work needing the main conversation	When delegated
Hooks	What must always or never happen: secrets, force-push, merge, gates, locked tests, releases	Advice; slow work	Every matching event; no context cost
Scripts	Deterministic logic shared by skills, hooks and CI	Judgement calls	When called
Connectors (MCP)	Confluence and Jira, Sentry, read-only MongoDB and schema DB, read-only Airflow, a browser	What CLIs do better: git, gh, test runners	Server at start; tools on demand
Plugins	Distribution and versioning of everything above the repo	Per-repo facts	Installed from the marketplace each repo names
Settings and policy	Permissions, sandbox, allowed marketplaces and MCP servers, minimum version, telemetry	Workflow logic	Always; managed settings win

MECHANISM	PUT HERE	KEEP OUT	LOADS
CI workflows	Independent verification and the stage triggers	The only copy of a rule	On GitHub events and schedules

Not every surface loads everything

Repo-declared plugins install in the terminal, the IDE, the desktop app, GitHub Actions (the `plugins` input) and the Agent SDK (the `plugins` option). They don't install in web sessions, scheduled tasks or Claude in Slack, and Slack sessions don't run repo hooks or load `.mcp.json`. For those surfaces, `tools/aisdlc.py vendor` copies the skills, agents, scripts and hooks into the repo's `.claude/` and writes the rules to `.claude/rules/`. Wherever hooks can't run, the CI required checks and branch protection are the gate, because they act on the pull request wherever it came from.

5

One builder, three repos

The builder is one subagent with one discipline: test-first, small commits, stop when stuck. Its stack expertise comes from versioned packs the repo selects, so a developer who mostly writes React can ship an Airflow change with the same guardrails, because the Airflow conventions, tests and checks arrive with the repo.

- **The manifest.** `.ai/repo.yaml` declares the stack, the check commands, how to run given test files, the JUnit paths, protected paths, plan exclusions and production release patterns. Hooks, scripts, agents and CI read the same file.
- **Path-activated stack skills.** `airflow-dag` loads on `dags/**` and `include/**`; `stack-mfe` depends on `stack-frontend`, so an MFE repo gets both.
- **Session context.** At start, resume and after compaction, the SessionStart hook injects the stack, the checks, the branch's Jira key, the state of intent, spec and plan, the evidence state and the rules.
- **Stories across repos.** The spec's repo-impact table sets the order: producers before consumers, each step backward compatible. Each repo gets its own branch, plan section, evidence and draft PR; `evidence.py` judges each repo only on its own row of ACs.

6

Plan: the PRD gate and the intent

`prd_lint.py` checks the same things every time: 15 sections (9 required), placeholders, Given/When/Then criteria covering every requirement, vague terms without numbers, NFRs without numbers, missing error, empty and no-access states, and unresolved open questions. The read-only `prd-analyst` subagent adds judgement: contradictions, roles and data scope, unhappy paths, data feasibility against schema snapshots, hidden cross-team scope and past incidents in the area.

On the example PRD, version 8 scored 16/100 with 3 blocking findings and 13 advisory ones. The product owner got concrete questions with proposed defaults they could accept in one word. Version 9 scored 97/100 and was READY.

1. The product owner labels the PRD ready. Confluence Automation sends `repository_dispatch` to GitHub.
2. `ai-prd-intake` fetches that page version as Markdown, lints it and runs the review headlessly with a typed verdict.
3. NEEDS_CLARIFICATION: a Confluence footer comment with the questions, a "Clarify PRD" Jira task for the product owner, and the epic moved to Needs Clarification. The gate re-runs on the next page version.
4. READY: `intent.md` is written from the PRD (problem, outcome, affected users and systems, constraints, open questions), linked to the exact page version, and the design stage starts in the same draft PR.

Intents also come from places other than PRDs: a Jira bug, a Sentry issue, a breached control band or a Slack thread. The `triage` skill writes those, with `status: draft` until their owner accepts them.

7

Design: context and the spec

SOURCE	CONNECTOR	MODE
Confluence, Jira	Atlassian Rovo MCP (OAuth); `atlassian.py` REST in CI, Cloud and Data Center	Read, plus comment, create task, transition
MongoDB	`mongodb-mcp-server@3.1.0 --readOnly --indexCheck` and a read-only user	Read-only, local or dev
Schema database	`@bytebase/dbhub@1.5.0` with `readonly = true` and a SELECT-only user	Read-only
Airflow	`astro-airflow-mcp@0.9.1` with `AF_READ_ONLY=true`	Read-only
Production errors	Sentry MCP; Datadog, Grafana or PagerDuty as alternatives	Read
Code and history	git, gh, rg (CLIs)	Read
Team knowledge	`knowledge/` and each repo's `docs/knowledge/`	Read

Committed schema snapshots come first; live connectors only fill gaps, and record values never enter the conversation. PRDs, tickets, logs and rows are untrusted data, never instructions, and no stage holds private data, untrusted content and an outbound channel at once.

The spec numbers requirements (`FR-n`) and acceptance criteria (`AC-n`), writes every AC as Given/When/Then with one observable outcome, designs only what changes, sequences the repos, plans one qualified test name per AC, sets a risk tier, and lists flagged concerns with their policy owners. A small, low-risk change takes the fast lane: summary, ACs, repo impact and test plan only. The gates stay the same.

8

Build: the plan, then the code

Work starts in plan mode, where Claude can read the codebase but not change it. The `plan` skill writes `plan.md` : the files that change (grouped by repo), the order of work as test-first tasks, the risks and the options not chosen, the proof for each AC, and any locked tests. It iterates until an engineer who never saw the conversation could implement the change from the plan alone.

- **Acceptance.** The engineer accepts routine plans in plan mode; for a risk: high spec the tech lead does, and the hook stops the agent accepting it. In CI, routine plans are accepted as auto mode for routine work.
- **The builder loop.** Load the stack skill, write the failing tests with qualified AC names, see them fail for the right reason, implement the minimum, run the checks, commit one task at a time and tick it in the plan. Three attempts per task, then stop and report.
- **Locked tests for bug fixes.** Reproduce the bug as a test, commit it alone, lock it with `evidence.py lock` , then fix the code. The hooks block edits to locked tests and `evidence.py` fails if their content changes, however it was edited.
- **Parallel work.** Tasks that touch different files run in separate worktrees (`claude --worktree`); start with two or three and add more only while review keeps up.
- **Plan drift.** When the work departs from the plan, the plan changes in the same commit. Evidence is incomplete while the diff touches a file the plan doesn't list.

9

Test: evidence and agent evals

`evidence.py run` runs the manifest's checks with hashed logs; links JUnit test names to this repo's ACs; runs new and changed tests on the merge-base to prove at least one fails there (strong when it fails on an assertion, weak when it only errored); checks that the plan is accepted, lists every changed file and that locked tests are unchanged; and writes a manifest for the commit on a clean tree. PASS is the only verdict that opens a PR.

LEVEL	WHAT HAPPENS	WHO TRUSTS IT
1 Local	The verify skill runs the evidence at a committed SHA; the `qa-verifier` subagent checks the tests prove their ACs	The PR hooks
2 CI re-run	`ai-evidence` re-runs the same script at the PR head on a clean runner, adds changed-line coverage, comments the verdict	Branch protection (required check)
3 Attested	The evidence bundle is uploaded and attested with Sigstore, tied to the commit	Reviewers and auditors

Agent evals. CLAUDE.md, skills, hooks and the plugins steer the agent, so they get regression tests too. Each repo keeps 20 to 50 real tasks in `evals/` (a prompt plus the checks that define acceptable). `agent_evals.py` runs each in a fresh worktree with `claude -p` ; `ai-agent-evals` runs them on every change to that

configuration and nightly, and fails below an 80% pass rate. Every production incident adds an eval. The standard's own skills are tested the same way with `claude plugin eval`, including a no-plugin baseline.

10

Deploy: review, approval gates and CI/CD

- **Review on every PR.** `REVIEW.md` defines three passes (bugs, security, compliance with the spec and plan), what Important means, a cap of five nits, and what not to report. The managed Claude Code Review reads it; the `ai-review` workflow does the same with claude-code-action. Findings never approve or block.
- **The fix loop.** `@claude` on a PR comment makes Claude address it and push to the PR. `pr-babysit` sweeps failing checks and open threads until only code-owner approval is left, and never merges.
- **Approval gates as hooks.** Draft PRs only; no merges; no pushes to protected branches. Production release commands are blocked unless the release manager has set `RELEASE_APPROVAL` for the session, and even then each one asks. Non-negotiable hooks go in managed settings, where developers can't switch them off.
- **CI/CD.** Claude starts with read-only judgement steps: `ai-evidence` triages its own failures and comments. Anything Claude writes arrives as a PR through branch protection. Deployments are exposed per environment, and production stays behind the release manager.

11

Maintain: closing the loop

Detection stays deterministic. `bands.py` keeps a rolling baseline of a metric (by default the CI test failure rate) and applies the Western Electric rules; `.ai/bands.yaml` sets what each tier allows:

TIER	CLAUDE MAY	OUTPUT
1 sigma	Nothing; the run is logged	A line in the job summary
2 sigma	Diagnose, read-only	An intent filed as an issue for the service owner
3 sigma	Also act through the routes listed: a draft PR, or a pre-approved runbook	A small fix through the normal review gate

Incidents can also arrive in Slack: Claude in Slack can take a bug thread and open a draft PR, with CI as the gate. The `learn` skill turns what each story taught into reviewed changes, moving a lesson up one rung each time it recurs:

RUNG 1
Knowledge entry, owned and dated

RUNG 2
A line in CLAUDE.md "Things Claude gets wrong"

RUNG 3
Rule or skill text

RUNG 4
Hook, lint or test

RUNG 5
An agent eval, for incidents

12

Ways to implement

SETUP	WHERE CLAUDE RUNS	STARTED BY	GATES ENFORCED BY	MOVE ON WHEN
1 Developer loop (start here)	Terminal, IDE, desktop	An engineer	Plugin hooks; people at each gate	10 to 20 stories measured against the baseline
2 CI evidence and review	GitHub runners; managed Code Review	Pull request events	Required check, branch protection, CODEOWNERS	The required check is stable
3 Stage-triggered automation	GitHub Actions	A labelled PRD, an accepted intent, an approved spec	Hooks in CI; PR approvals	Lead time falls and rework doesn't rise
4 Cloud, parallel and Slack	Anthropic-hosted sandboxes	Web, phone, Slack threads	Vendored hooks on the web; CI for Slack-made PRs	Review capacity becomes the limit
5 Closed loop	GitHub Actions or scheduled tasks	Metrics and schedules	Band tiers; review gate	Intent volume outgrows triage
6 Orchestrated at scale	Agent SDK services or Managed Agents	Your workflow engine	SDK hooks and callbacks; the same CI gates	The end state

```
# one-time per repo (from a checkout of the standard)
python3 tools/aisdlc.py init   --target ../data-airflow --stack airflow  --jira PAY
python3 tools/aisdlc.py init   --target ../payroll-mfe  --stack mfe      --jira PAY
python3 tools/aisdlc.py init   --target ../web-shell    --stack frontend --jira PAY
python3 tools/aisdlc.py doctor --target ../data-airflow
# then, in the repo, with Claude Code
/sdlc-core:prd-to-pr https://acme.atlassian.net/wiki/spaces/PAY/pages/123456789
```

13

Claude Code features and how the standard uses them

FEATURE	HOW THE STANDARD USES IT
CLAUDE.md and `.claude/rules/`	Repo facts under a page; vendored always-on rules for surfaces without hooks
Skills	Sixteen skills; `paths` loads stack skills automatically; `allowed-tools` runs bundled scripts without prompts; `disable-model-invocation` on the orchestrator
Subagents	Five agents; `disallowedTools` keeps four read-only
Hooks	SessionStart for context and rules; PreToolUse guards for shell, edits and MCP PR creation; Stop for the verify reminder; exec form, fail closed
Plan mode, worktrees	The plan stage; parallel tasks and stories
Plugins and marketplace	Dependencies between plugins, per-repo enablement, auto-update, semver tags

FEATURE	HOW THE STANDARD USES IT
`claude plugin` CLI	`validate --strict` in CI, `details` for the token budget, `eval` with a no-plugin baseline, `tag` for releases
MCP	OAuth SaaS servers in the connectors plugin; read-only data servers per repo
Permissions, sandbox, managed settings	Repo allow and deny lists; a standard and a regulated policy tier; minimum version; telemetry
Headless mode and claude-code-action	`claude -p --json-schema` for typed verdicts; the action's `plugins` input in every workflow; @claude fixes
Code Review	The managed reviewer reads `REVIEW.md`; severities Important, Nit and Pre-existing
Web sessions, scheduled tasks, Claude in Slack	Setups 4 and 5, with vendored configuration
Agent SDK, Managed Agents, dynamic workflows	Setup 6: the same plugins in your services, hosted long-running agents, multi-repo fan-out
OpenTelemetry	Sessions, PRs, commits, cost and tokens, plus hook decisions, for the metrics in section 15

14

Running it in an enterprise

Distribution and updates

Plugins update automatically from the marketplace (running sessions need `/reload-plugins`). Vendored repos re-run `aisdlc.py vendor` and review the diff; the version is pinned in `.claude/aisdlc/VERSION.json`. CI workflows pin `AISDLC_REF` to a release tag. Each plugin uses semver: patch for wording, minor for a new skill or check, major for a changed gate. CODEOWNERS give each stack pack to its guild.

Two policy tiers

`policy/managed-settings.json` registers the marketplace, force-enables the core plugins, allows only this marketplace and the vetted MCP servers, forbids bypass mode, sets a minimum Claude Code version and exports telemetry. `managed-settings.regulated.json` adds managed-only permission rules and hooks, an OS sandbox that refuses to start when unavailable, an egress allowlist, credential isolation and no side-loaded plugins.

Security model

- **Least privilege by stage.** Analysts and reviewers are read-only; connectors are read-only against local or dev data; the builder writes only to its own branch; PR creation is gated; only people merge and release.
- **Untrusted input stays data.** PRD, ticket, log and record text never becomes instructions.
- **A deterministic floor.** Hooks block secrets, force-pushes, merges, infrastructure and cluster changes, data-store writes, `curl | sh`, environment dumps and unapproved releases, and fail closed.
- **Workflows that untrusted users can trigger hold no secrets** beyond what the action needs.
- **Audit trail.** The artifact chain in git, CI run links, attestations and OpenTelemetry events.

Roles

ROLE	OWNS
Product owner	The PRD and the intent; answers gate questions or accepts the defaults
Tech lead	Spec approval, risk tier, high-risk plans, cross-repo sequencing
Engineer	Runs the loop, accepts routine plans, reviews the AI's PR before others, merges with code owners
Release manager	Production authorisation
Service owner	Triage of what the loop finds; the control bands
Platform team	The standard, connectors, CI templates, metrics, releases, evals
Security	Managed policy, the connector allowlist, high-risk approvals

Cost

Measured with `claude plugin details` on Claude Code 2.1.308, the always-on context cost is about 1,763 tokens for sdlc-core, 278 for the connectors and 149 to 181 per stack pack. Skill and agent bodies load only

when used. A behavioural eval case cost about $0.10 per run. Track cost per merged PR from OpenTelemetry.

15

Rollout and metrics

WEEKS	DO	MOVE ON WHEN
1-4	Setup 1 in one repo per stack; baseline the last 20 comparable stories; PRD gate on every new PRD	The PRD gate and evidence ran on 3 stories
3-6	Setup 2: required evidence check, REVIEW.md, Code Review or ai-review	The required check is stable for 2 weeks
5-10	Setup 3 for one product area	Lead time below the baseline median
7-12	Setup 4: parallel cloud sessions, Slack bug intake	Review time per PR isn't rising
9-14	Setup 5: bands, triage, 20 agent evals nightly	The first intent from a band was triaged
13+	Setup 6 only if volume demands it	

Each stage has a leading and a lagging measure, taken from the artifacts' own timestamps: time from PRD to READY and the bounce rate; hours from intent to spec and spec changes after the plan; first-pass merges and how often the diff matches the plan; first-pass CI success and eval pass rate, review time and change failure rate; time to first review and escaped defects; time from a band breach to an intent, and repeat incidents. Don't target the share of AI-written code or PR counts: DORA 2025 found AI raises throughput and instability together.

16

What others do

A summary of the sourced survey; the interactive page has 29 cases with filters, charts and links.

COMPANY	WHAT THEY DO	LESSON ADOPTED HERE
Anthropic	Code changes merged per day rose from about 2,500 (May 2026) to over 30,000 (September); AI review on every PR	Scale review, source control and CI before scaling agents
Intercom	A governed Claude Code plugin marketplace; over 93% of PRs agent-driven; auto-approval only for small, low-risk, logged PRs	Treat the agent's configuration as a product
HubSpot	Headless Claude Code on Kubernetes; 7,000+ AI PRs merged; a Stop hook refuses "done" until the build is green	Deterministic steps out of the prompt
Spotify	Honk on the Claude Agent SDK; merged changes 8,100 to 17,000 year on year; verifiers in the stop hook	Verify before a PR exists; data repos need tests first
Stripe	Minions: one-shot agents, more than 1,300 merged PRs a week, all human-reviewed; at most two CI rounds	Bounded iteration, then a human
Wix	AirBot turns Airflow failures into fix PRs; 675 to about 850 engineering hours saved a month	Read-only analysis first, human-approved writes
Sentry, Carvana, CircleCI	Root-cause analysis to Claude-written PRs; alerts handled in Slack; CI failures fixed by an agent whose PRs CI already passed	Give the agent the diagnosis; CI is the referee
Meta, Uber	AI-written tests filtered by objective criteria; mutation testing rejects change-detector tests	Validate the tests, not just green CI
Google, Airbnb, LG CNS	Migrations as pipelines of deterministic steps with agent steps inside; review became the bottleneck	Cap change volume at what reviewers absorb

17

What doesn't work

- **Building the orchestrator first:** months of infrastructure before learning whether the gates and evidence work.
- **One agent, one giant prompt, no gates:** agents drift from specs. Use a deterministic skeleton with agent steps.
- **Prose-only rules for things that must hold:** the spec-gate eval scored 0.5 without the hooks and 1.0 with them.
- **Letting agents change tests to get green:** block skip markers, lock bug-fix tests, require fail-to-pass.
- **Unbounded retries:** cap them and escalate with what was tried.
- **Optimising for the share of AI code or PR counts:** instability and review time rise.
- **Large PRs:** one story per branch, small tasks, a PR per repo.
- **Ever-growing instructions:** keep CLAUDE.md under a page and always-on rules near 50 lines.
- **Production data in prompts:** use snapshots, read-only dev databases and synthetic fixtures.
- **Auto-merge before the pilot proves quality,** and skipping the baseline.

18

What was verified

In this repository, 22 October 2026

- **Pass** 96 unit tests on Python 3.9 and 3.13: PRD lint, AC coverage, fail-to-pass in throwaway git repos, evidence (including plan drift and locked tests), the agent-evals runner with a stub CLI, control bands, every hook (including the plan gate, locks and release gate), the Atlassian client, schema snapshots, the onboarding CLI and manifest consistency.
- **Pass** `claude plugin validate --strict` for the marketplace and all five plugins (Claude Code 2.1.308).
- **Pass** `make demo`: PRD gate, cross-stack AC coverage, fail-to-pass and plan evidence, the PR gate going from deny to allow, a locked test and the release gate.
- **Pass** Behavioural evals with real model runs (see `docs/evals.md`).
- **Pass** Workflow, JSON and YAML templates parse; the pages render without console errors or horizontal overflow at desktop and phone widths in both themes.

Check in your environment

- **To do** Live calls to your Confluence, Jira, Sentry, MongoDB and schema database.
- **To do** The GitHub workflows on real runners, including installing a private marketplace in CI.
- **To do** Web sessions, scheduled tasks and Claude in Slack with vendored configuration.
- **To do** The policy tiers on real machines (the sandbox needs bubblewrap and socat on Linux).
- **To do** Replace the placeholder CODEOWNERS teams and the example Jira statuses.

AI SDLC standard 2.0 for Claude Code · source: amarkum/prd-to-pr-rule-skills

Stages from Anthropic's AI-native SDLC playbook